

The Complexity Shield: Modeling and Measuring the Impact of Mock Sharing Patterns

Gengwu Zhao, *Student Member, IEEE*, Lu Xiao, *Member, IEEE*, Xinyi Li, *Student Member, IEEE*, and Sunny Wong

Abstract—

Background: Unit testing relies heavily on mocking to isolate dependencies, yet the structural organization and reuse of mocking logic remain largely unexplored. While mock duplication (mock clones) is known to be prevalent, there has been no foundational taxonomy to categorize how developers share and reuse these objects across test suites.

Objective: This study aims to establish a formal understanding of “mock sharing patterns”—the strategies used to organize mocking logic locally within tests or share them via helper methods and setup routines—and to quantify their impact on test code complexity.

Methodology: We analyzed 56,502 mock objects from 97 mature Apache Software Foundation projects. We introduced a three-level taxonomy of mock reuse: *L0* (non-shared), *L1* (fully shared), and *L2* (partially shared). The study employed two foundational engines: a Mapping Pipeline for classification and a Conversion Simulation engine to model “upgrades” and “downgrades” between reuse levels. Impact was measured using Cognitive Complexity for Test Readability (CCTR), a metric sensitive to mocking-related constructs.

Results: (1) Distribution (RQ1): Mock reuse is uncommon at the project level, with most systems relying predominantly on local *L0* mocks. However, as mocking tasks grow in complexity (five or more stubs), developers reliably shift toward partial reuse (*L2*). (2) Evolution (RQ2): Sharing practices are remarkably stable; 38% of projects maintain consistent levels over time, while only 22% show systematic movement toward greater reuse. (3) Complexity (RQ3): Simulation reveals that upgrading impactful non-shared mocks to shared configurations can reduce per-test complexity by 14–67%. Conversely, existing shared mocks serve as a “complexity shield,” preventing a 24–160% spike in complexity that would occur if they were abandoned.

Conclusion: The study provides quantitative evidence that mock sharing strategies directly influence test maintainability. It highlights a significant “improvement gap” where automated tools could identify high-impact sharing opportunities to substantially reduce project-wide technical debt.

Index Terms—Code Clone, Mocking, Unit Testing, Refactoring, Test Complexity

I. INTRODUCTION

Unit testing is fundamental to modern software quality assurance, providing fast feedback on the correctness of individual components and supporting long-term maintainability [1], [2], [3]. A key challenge in unit testing is *dependency isolation*—the need to test a component without invoking its real

external dependencies, such as databases, network services, or file systems [4]. Among the techniques developed for this purpose, *mocking* has emerged as the dominant approach: mock objects replace real dependencies with programmable substitutes whose behavior can be precisely controlled within the test [5]. Mocking frameworks—most notably Mockito—are widely adopted in real-world Java projects and play an essential role in crafting deterministic and maintainable tests [6], [7]. Empirical studies have confirmed this widespread adoption: [8] found that Mockito is used in 74% of Apache projects that employ a mocking framework, while [9] and [10] further documented the prevalence and diversity of mocking practices across open-source communities.

However, as test suites grow in size and complexity, maintaining high-quality test code becomes increasingly costly [11], [12]. Test code is subject to the same maintenance challenges as production code—including code duplication, structural decay, and accumulating technical debt [13], [14], [15]. The phenomenon of *test smells*—structural anti-patterns that hinder test comprehensibility and maintainability—has been extensively documented [16], [17], [18], yet existing research on test quality has focused primarily on general fixture organization [19] and assertion-level smells [13], with comparatively little attention devoted to the specific challenges introduced by mocking logic.

Recent work has begun to expose these challenges. [20], [21] conducted the first systematic investigation of duplicated mocking logic in Java test code. Through an empirical study across six open-source projects, they introduced the concept of *mock clones*—repeated sequences of mock creation and stubbing logic scattered across test cases—and demonstrated that 71%–100% of frequently mocked classes contain such duplication [20]. These studies establish that mock duplication is both prevalent and amenable to elimination, but a foundational question remains unanswered: *How do developers structure and organize mocking logic in the first place?*

In practice, mock objects may be created and configured entirely within individual test methods, or their setup logic may be reused across tests through shared setup routines (e.g., `@BeforeEach` methods) or helper methods. These different organizational strategies—referred to here as *mock sharing patterns*—have not been systematically studied at scale, despite their direct relevance to test maintainability. Prior work on test fixture strategies has examined how developers organize general setup code [4], [22], [23], and the AAA (Arrange-Act-Assert) pattern literature has shown that the Arrange phase is typically the most complex and variable

Manuscript received XXXX XX, 2026; revised XXXX XX, 2026.

G. Zhao, L. Xiao, and X. Li are with the School of Systems and Enterprises, Stevens Institute of Technology, Hoboken, NJ 07030 USA (e-mail: gzhao9@stevens.edu; lxiao6@stevens.edu; xli221@stevens.edu).

S. Wong is with Envestnet, Inc., Berwyn, PA 19312 USA (e-mail: sunny@computer.org).

portion of a test [24], [25]. However, these studies address general fixture duplication and do not examine the specific structure or reuse patterns of mocking logic within the Arrange phase.

Understanding mock sharing is important because mocking logic often constitutes a substantial portion of the Arrange phase and thus directly affects test readability and complexity [8], [10]. [26] recently proposed CCTR (Cognitive Complexity for Test Readability), a test-specific complexity metric that explicitly incorporates mocking constructs as an independent complexity dimension. Traditional metrics such as McCabe’s Cyclomatic Complexity [27] and SonarSource’s Cognitive Complexity [28] were designed for production code and assign near-zero scores to the vast majority of test methods, making them unable to differentiate between tests of varying structural quality [26]. The existence of such a mocking-aware metric creates a unique opportunity to investigate whether the way mocking logic is organized—shared versus non-shared—measurably influences test complexity.

Despite the growing body of work on mocking practices, a critical gap persists. Existing empirical studies have characterized mocking framework adoption [8], [9] and mocking decisions [7], [10], [29], but they do not examine how mocking logic is *shared* (and thus reused) across test cases. Research on test fixtures has studied general setup sharing strategies [4], [22], [23], but has not isolated the specific contribution of mocking logic. Studies on mock clones [20], [21] have demonstrated that mock duplication is pervasive, but they focus on detection and elimination rather than on the underlying organizational taxonomy. No prior work has systematically examined how mock sharing patterns are distributed across projects, how they evolve over time, or how they correlate with test complexity.

To fill this gap, this paper presents the first large-scale empirical study of mock sharing patterns across 260 Apache Software Foundation projects—a diverse ecosystem spanning distributed systems, databases, messaging middleware, and developer tooling [30]. We introduce a three-level taxonomy of mock reuse (L0: non-shared, L1: fully shared, L2: partially shared) and analyze 56,502 mock objects across 97 projects that meet our inclusion criteria. Our goal is to establish a foundational understanding of the distribution, evolution, and complexity implications of mock sharing. This work also serves as a critical stepping stone for subsequent research on automated mock clone detection and refactoring [21].

We investigate the following research questions:

- **RQ1:** How are mock sharing levels (L0–L2) distributed across projects?
- **RQ2:** How do mock sharing levels change over time within projects?
- **RQ3:** How do different mock sharing levels correlate with test complexity (CCTR)?

Our findings reveal that mock reuse is uncommon at the project level, with most projects relying heavily on locally defined mocks; however, developers reliably shift toward partial reuse when mocking tasks become structurally complex. Longitudinal analysis shows that sharing practices tend to remain stable over project lifetimes, with systematic adoption

of shared patterns occurring in only 22% of projects. Through conversion-based simulation, we demonstrate that upgrading non-shared mocks to shared configurations reduces per-test complexity by 14–67% for the most impactful cases, while abandoning existing sharing would increase complexity by 24–160%. These results provide concrete, quantitative evidence that mock sharing strategy directly impacts test maintainability, and they highlight substantial untapped opportunities for complexity reduction through systematic mock reuse.

In summary, we present the first large-scale empirical study to model and measure the structural impact of mock sharing patterns. Our contributions are four-fold:

- **A Formal Taxonomy of Mock Reuse:** We establish a three-level model (*L0*, *L1*, *L2*) to categorize the degree of sharing in mock configurations, providing a rigorous framework for studying mock organization.
- **Longitudinal Analysis of Evolutionary Mechanics:** We trace 56,502 mock objects across nearly a decade of project history to identify the forces—such as *Redundancy Drift* and *Structural Inertia*—that govern the decay or consolidation of mock sharing over time.
- **The “Complexity Shield” Measurement:** Using the Cognitive Complexity for Test Readability (CCTR) metric, we quantify the “shielding” effect of shared mocks. We demonstrate that existing shared configurations prevent a complexity spike of up to 160% per test, acting as a vital architectural defense.
- **Quantification of the Improvement Gap:** Through conversion-based simulation, we measure a 56% “improvement gap” in the ecosystem, identifying substantial untapped potential to reduce aggregate project complexity by 20–60% through targeted shared-first accretion.

II. BACKGROUND

A. Mocking in Unit Testing

Unit testing is fundamental to modern software quality assurance. By focusing on the correctness of individual functions or classes in isolation, unit tests provide fast feedback, help localize defects, and support long-term maintainability [1], [2], [31]. Test-driven development practices further underscore the central role of unit tests as a design and verification tool throughout the software lifecycle [32]. A key requirement for effective unit testing is *dependency isolation*—the ability to test a component without invoking its real external dependencies, such as databases, network services, or file systems [4], [6]. Among the techniques developed for this purpose, *mocking* has become the dominant approach: mock objects replace real dependencies with programmable substitutes whose behavior can be precisely controlled within the test [5].

Mocking frameworks have evolved significantly over the past two decades. Freeman et al. [5], [33] introduced jMock, establishing key principles for responsibility-based design and interaction testing. EasyMock [34] followed with a simpler API for dynamic proxy-based mocking. Today, Mockito [35] has become the de facto standard in Java ecosystems due to its expressive and readable API. Empirical studies have confirmed

this dominance: Xiao et al. [8] found Mockito used in 74% of Apache projects that adopt a mocking framework, while Mostafa and Wang [9] and Spadini et al. [7], [10] further characterized mocking practices across diverse open-source communities. Mocking has also been extended to specialized domains, including embedded systems [36], [37], cloud platforms [38], and mobile applications [39], [40]. Supporting tools such as MockSniffer [29] and automated test generation frameworks [41] further assist developers in managing mock usage.

Mock objects generally follow a *create–stub–verify* lifecycle, as illustrated in Listing 1. A mock object is first *created* to replace a real dependency (Line 2); its behavior is then *stubbed* by specifying return values for particular method invocations (Lines 4–5); finally, after the act phase, the test *verifies* that the mock was called in the expected manner (Line 7).

```

1 @Test
2 public void testCreateOrder() {
3     User user = mock(User.class);           // Create:
4     // mock the dependency
5     Order order = new Invoke(user);
6     when(user.getName()).thenReturn("Alice");
7     // Stub: define
8     when(user.getTier(any())).thenReturn("BASIC");
9     // expected behavior
10    order.getOrderID(user);                  // Act:
11    // invoke target method
12    verify(user).getName();                  // Verify:
13    // check interactions
14 }

```

Listing 1: A Mockito Create–Stub–Verify Example

While mocking enhances testability, it also introduces substantial setup logic. In practice, a single test case may require configuring multiple mocks, each with several stubbed methods, leading to verbose and often redundant Arrange logic across test suites. Zhao et al. [20] conducted the first systematic empirical investigation of this phenomenon across six open-source Java projects, introducing the concept of *mock clones*—repeated sequences of mock creation and stubbing logic scattered across test cases—and demonstrating that 71%–100% of frequently mocked classes contain such duplication. This perspective aligns with the broader principle that structural reorganization of duplicated code—even when behavior-preserving—can yield substantial maintainability improvements [42].

B. Test Code Complexity

Measuring the complexity of test code is essential for assessing its maintainability and comprehensibility. Traditional complexity metrics such as McCabe’s Cyclomatic Complexity [27], SonarSource’s Cognitive Complexity [28], and composite indices like the Maintainability Index [43] were originally designed to evaluate functional code by penalizing nested control structures, logical operators, and flow-disrupting constructs. While these metrics are effective for production code, they are ill-suited for unit tests, as test methods usually do not involve deep control flow. As a result, these metrics assign near-zero scores to the vast majority of test methods—even

those that are structurally complex or difficult to understand—rendering them unable to meaningfully differentiate between tests of varying quality [26]. This limitation is particularly concerning given that test smells—structural indicators of maintainability problems—have been shown to accumulate and persist over project lifetimes [15], [44]. Simpler size-based metrics such as Lines of Code (LOC) similarly fail to capture the structural organization of test logic, as they treat all lines equally regardless of their semantic role within the test [45].

The AAA (Arrange-Act-Assert) pattern provides a structural framework for understanding test organization [46]. Wei et al. [24] developed automated approaches to identify AAA segments and empirically analyzed how developers structure test cases, finding that the Arrange phase is often the most complex and variable portion of a test. Within this Arrange phase, mocking logic plays a particularly significant role: depending on how mock objects are organized, a test may contain extensive inline mock creation and stubbing, or may delegate this logic to shared helper methods. These different forms of mock organization result in substantially different distributions of mocking constructs within individual test methods, and consequently different levels of per-test complexity.

Ouédraogo et al. [26] proposed CCTR (Cognitive Complexity for Test Readability), a test-specific metric that addresses the limitations of existing approaches. CCTR is defined as:

$$\text{CCTR} = \alpha \cdot N + \beta \cdot A + \gamma \cdot M + \delta \cdot T \quad (1)$$

where N denotes control-flow nesting complexity following the original SonarSource definition; A counts assertion and `fail()` statements; M captures mocking-related constructs including calls to `mock()`, `when()`, and `verify()`; and T represents annotation-based signaling, assigning +1 for common annotations such as `@Test` and `@BeforeEach`, and +2 for specialized annotations such as `@ParameterizedTest`. All weights are set to $\alpha = \beta = \gamma = \delta = 1.0$, reflecting a design goal of simplicity and interpretability.

CCTR is particularly well-suited for our study for two reasons. First, by explicitly incorporating mocking constructs as an independent complexity dimension through the M term, CCTR is sensitive to exactly the variation we investigate: when mocking logic is written inline within each test method, the M contribution to per-test CCTR is high; when the same logic is extracted into shared setup methods, the per-test M contribution decreases. Traditional metrics such as Cyclomatic Complexity and LOC cannot distinguish between these two organizational forms, as they are blind to the semantic role of mocking invocations. Second, CCTR has been validated at scale across 15,750 test suites generated by EvoSuite, GPT-4o, and Mistral Large, demonstrating that it effectively discriminates between test suites of different structural quality and produces scores that better reflect developer-perceived comprehension effort compared to existing metrics [26].

We therefore adopt CCTR as our complexity measure to quantify how different forms of mock organization affect the cognitive effort required to understand individual test cases.

III. MODELING MOCK SHARING PATTERNS AND CONVERSION SEMANTICS

In this section, we establish a formal model to characterize how mock objects are reused across test cases. We begin by decomposing the lifecycle of a mock into its fundamental operations—creation, stubbing, and verification—and classifying each based on its usage scope. Drawing on an empirical analysis of 260 Apache projects, we prune this theoretical state space to focus on the dimensions that provide the most significant discriminative power in practice. This leads to the definition of three distinct Mock Share Levels (L0, L1, and L2), which categorize mocks based on the homogeneity of their shared logic. Finally, we define the conversion semantics that govern how mocks transition between these levels. By modeling these “upgrades” and “downgrades,” we provide a rigorous mechanism for measuring the impact of different mock share strategies on the overall complexity and maintainability of the test code.

A. Step-Level Sharedness Modeling: Create, Stub, and Verify

The overall reuse level of a mock object is determined by the collective sharedness of its individual lifecycle steps: creation, stubbing, and verification. We characterize each mocking statement within these steps based on its usage scope:

- **Shared:** A statement located within shared setup routines (e.g., `@BeforeEach`) or helper methods, making its logic accessible to multiple test cases.
- **Local:** A statement defined entirely within a single test method, making its logic accessible only to that specific test case.

a) **Atomic State Space.** By classifying the scope of every statement associated with a mock, we define the *atomic state space* for each lifecycle step—Creation (*C*), Stubbing (*S*), and Verification (*V*)—as a mapping to its possible sharedness states:

- $Space(C) = \{shared, local\}$
- $Space(S) = \{fully-shared, partially-shared, local-only, none\}$
- $Space(V) = \{fully-shared, partially-shared, local-only, none\}$

The variation in the number of states per dimension reflects the different structural cardinalities of each step. Creation (*C*) is binary because a mock object is instantiated at exactly one site; thus, the sole creation statement is either local or shared. In contrast, stubbing (*S*) and verification (*V*) are *sets of statements* that may contain zero, one, or multiple elements. That is, a mock object may have multiple *stubs* and multiple *verifies*. Thus, the sharedness of these dimensions is determined by the composition of their respective sets:

- **Fully-shared:** Every statement in the set is a *shared statement*.
- **Partially-shared:** The set is heterogeneous, containing at least one *shared statement* and at least one *local statement*.
- **Local-only:** Every statement in the set is a *local statement*.
- **None:** The set of statements is empty.

TABLE I
DISTRIBUTION OF SHAREDNESS FOR MOCK LIFECYCLE STEPS (ACROSS 260 PROJECTS)

Step	Sharedness	#Mocks	Percentage
Creation			
	Local	26,406	44%
	Shared	33,041	56%
Stubbing			
	Local only	25,590	43%
	Partially shared	10,494	18%
	Fully shared	6,113	10%
	None	17,250	29%
Verification			
	Local only	1,840	3%
	Partially shared	0	0%
	Fully shared	19	0%
	None	57,588	97%

b) **Empirical Pruning of the State Space.** To refine our theoretical model for practical application, we conducted data mining across 260 Apache projects to populate the dimensions of our proposed state space. The results, summarized in Table I, provide a large-scale snapshot of how creation, stubbing, and verification are distributed in real-world test suites.

Table I reveals distinct behavioral patterns for each lifecycle step. **Creation** is relatively balanced, with 56% of mocks instantiated in shared locations compared to 44% created locally. **Stubbing** exhibits the most significant structural diversity: while 43% of mocks rely solely on local stubs, a substantial 28% utilize either fully or partially shared stubbing logic, and 29% contain no stubbing at all. This variation confirms that stubbing structure provides the necessary discriminative power to characterize complex reuse behaviors.

In contrast, **verification** contributes negligible informative variation: 97% of the analyzed mocks contain no verification statements, and shared verification instances are nearly non-existent. The implication is that, in the context of Apache projects, the community tend to use mocks primarily as data-providers (stubbing) rather than interaction-checkers (verifications). Guided by these findings, we made the decision to **prune** the state space. While our analysis pipeline extracts verification data, we formally exclude the verification dimension (*V*) from the construction of our mock reuse levels. By focusing exclusively on the interplay between creation (*C*) and stubbing (*S*), we ensure that the patterns in Section III-B targets the structural attributes that most meaningfully impact mock organization in practice.

B. Mock Share Levels: L0, L1, and L2 Patterns

Following the empirical pruning of the state space above, a mock object’s integrated reuse structure is determined by the specific mapping of its *creation* (*C*) and *stubbing* (*S*) states. While theoretically multiple combinations exist within the Cartesian product of these dimensions, not all are structurally feasible. Because a mock instantiated within a local test method cannot be referenced by external shared setup routines, the states *local creation* + *fully-shared stubbing* and *local creation* + *partially-shared stubbing* are impossible.

Consequently, any form of shared behavioral configuration ($S \neq \text{local-only}$) necessitates that the mock be created in a shared location.

We synthesize the remaining feasible states into three mutually exclusive levels. Our classification is guided by three principles that distinguish how mock logic is shared: 1) The reuse level is determined by the collective state of *both* creation and stubbing. A mock is only as shared as its most local component. 2) A mock reaches the highest level of reuse (*fully shared*) only if 100% of its available lifecycle logic—both its instantiation and every defined behavior—resides in shared code. 3) We recognize a middle tier (*partially shared*) where the lifecycle is split, allowing a shared core to be reused while preserving the flexibility of test-specific local extensions.

These principles yield the following formal definitions for our taxonomy:

a) Level 0 (L0): Non-Shared Mocks.:

$$C = \text{local} \quad \text{and} \quad S \in \{\text{local-only}, \text{none}\}$$

At this level, all available mock configuration occurs in a single test method, with no reuse across the test suite.

b) Level 1 (L1): Fully Shared Mocks.:

$$C = \text{shared} \quad \text{and} \quad S \in \{\text{fully-shared}, \text{none}\}$$

All available mock configuration is centralized in shared code. This includes cases where a mock has shared creation but no stubbing; since its entire available definition is defined in shared code, it is considered fully reused.

c) Level 2 (L2): Partially Shared Mocks.:

$$C = \text{shared} \quad \text{and} \quad S = \text{partially-shared}$$

This represents a hybrid reuse strategy where a shared core configuration is reused across tests, while additional stubs are introduced locally within specific test methods to satisfy test-specific requirements.

d) Illustrative Examples.: Listing 2 provides concrete examples of how these levels manifest in Java test code, illustrating the distinct structural signatures of each level:

- **L0:** In both L0 examples (left column, top and bottom examples), the mock object is entirely self-contained within the test method. In the top-left case, both the mock’s identity (ID) and its behavioral state (Status) are defined locally. In the bottom-left case, even though two different tests share the same identity “TX-100”, the configuration is duplicated locally in each, representing a total lack of architectural reuse.
- **L1:** The L1 example (top-right) demonstrates a *fully shared* configuration. Here, the `createStandardOrder()` helper method encapsulates the entire lifecycle of the mock. Because

the helper returns a “ready-to-use” object, the test methods are stripped of all configuration logic, achieving maximum centralization.

- **L2:** The L2 example (bottom-right) illustrates a *partially shared* or hybrid approach. The `mockBaseOrder()` helper provides a shared “core” identity (the ID “TX-100”), but it does not define the specific functional behavior. Instead, individual tests extend this base by stubbing unique methods—`getItems()` for subtotal calculations and `getWeight()` for shipping calculations—allowing the mock to be reused while remaining flexible to specific test requirements.

C. Theoretical Mock Share Levels Conversion Semantics

To examine the impact of different mock organization strategies, we define formal conversion semantics that describe how a mock’s reuse structure is transformed between the non-shared level (L0) and the shared levels (L1 and L2). These conversions relocate mock creation and stubbing logic between test-local and shared architectural scopes without altering the observable test behavior.

a) Transformation Directions: Upgrade and Downgrade.: We define two symmetric transformations, which can be visualized by moving horizontally between the examples in Listing 2:

- **Upgrade ($L0 \rightarrow L1/L2$):** An upgrade represents the process of *extracting* common mocking logic into shared code.
 - **To L1:** As seen in the transition from the top-left to top-right of Listing 2, this involves moving 100% of the mock’s statements into a helper like `createStandardOrder()`. This identifies the untapped potential for total configuration reuse.
 - **To L2:** As seen in the transition from the bottom-left to bottom-right, only the “core” identity (e.g., `getId()`) is extracted into `mockBaseOrder()`, while functional stubs (e.g., `getItems()`) remain in the test. This identifies the potential for template-based reuse.
- **Downgrade ($L1/L2 \rightarrow L0$):** A downgrade represents the process of *inlining* shared configuration back into individual tests. This effectively reverses the examples in the right column of Listing 2 back into the duplicated local statements seen on the left. This conversion is used to measure the maintenance benefit provided by the existing shared architecture; by removing the abstraction, we expose the redundant complexity that sharing was previously concealing.

b) Conversion Constraints and Integrity.: Conversions are defined specifically between L0 and the shared levels. We do not consider direct conversions between L1 and L2, as they represent different design intents: L1 denotes an immutable, fully-encapsulated configuration, while L2 explicitly preserves test-specific flexibility.

All transformations are strictly *behavior-preserving* at a logical level. Regardless of whether a mock is configured via a shared helper or local statements, its interactions with

L0: Local (for L1 Upgrade)

```

1 @Test
2 void testProcessOrder() {
3     Order mockOrder = mock(Order.class);
4     when(mockOrder.getId())
5         .thenReturn("TX-100");
6     when(mockOrder.getStatus())
7         .thenReturn("OPEN");
8     assertTrue(processor.isValid(mockOrder));
9 }
10 @Test
11 void testLogOrder() {
12     Order mockOrder = mock(Order.class);
13     when(mockOrder.getId())
14         .thenReturn("TX-100");
15     when(mockOrder.getStatus())
16         .thenReturn("OPEN");
17     logger.record(mockOrder);
18 }

```

L1: Fully Shared

```

1 private Order createStandardOrder() {
2     Order mockOrder = mock(Order.class);
3     when(mockOrder.getId())
4         .thenReturn("TX-100");
5     when(mockOrder.getStatus())
6         .thenReturn("OPEN");
7     return mockOrder;
8 }
9 @Test
10 void testProcessOrder() {
11     assertTrue(
12         processor.isValid(createStandardOrder())
13     );
14 }
15 @Test
16 void testLogOrder() {
17     logger.record(createStandardOrder());
18 }

```

L0: Local (for L2 Upgrade)

```

1 @Test
2 void testOrderSubtotal() {
3     Order mockOrder = mock(Order.class);
4     when(mockOrder.getId())
5         .thenReturn("TX-100");
6     when(mockOrder.getItems())
7         .thenReturn(items);
8     assertEquals(50.0, calc.subtotal(mockOrder));
9 }
10 @Test
11 void testOrderShipping() {
12     Order mockOrder = mock(Order.class);
13     when(mockOrder.getId())
14         .thenReturn("TX-100");
15     when(mockOrder.getWeight())
16         .thenReturn(10.5);
17     assertEquals(15.0,
18         calc.shipping(mockOrder));
19 }

```

L2: Partially Shared

```

1 private Order mockBaseOrder() {
2     Order mockOrder = mock(Order.class);
3     when(mockOrder.getId())
4         .thenReturn("TX-100");
5     return mockOrder;
6 }
7 @Test
8 void testOrderSubtotal() {
9     Order order = mockBaseOrder();
10    when(order.getItems()).thenReturn(items);
11    assertEquals(50.0, calc.subtotal(order));
12 }
13 @Test
14 void testOrderShipping() {
15     Order order = mockBaseOrder();
16    when(order.getWeight()).thenReturn(10.5);
17    assertEquals(15.0, calc.shipping(order));
18 }

```

Listing 2: Taxonomy of Mock Reuse: Conversion from Level 0 to Level 1 and Level 2

the System Under Test (SUT) remain semantically identical. It is important to note that these conversions represent theoretical structural shifts intended to isolate the impact of logic relocation; as such, they do not account for pragmatic implementation constraints, such as Java language syntax limitations (e.g., variable visibility, checked exceptions, or final modifier restrictions) that might complicate a physical refactoring. By simulating these transitions in the abstract, we can directly compare the complexity of alternative mock organizations within the same testing context, independent of the incidental boilerplate required to implement them.

IV. RESEARCH QUESTIONS

To build a foundational understanding of how mocks are structured and used in practice, and how these structures may affect test maintainability, we examine the three mock sharing levels defined in the previous section. Our goal is to characterize their prevalence, how they evolve as projects grow, and whether they meaningfully influence test complexity. Toward this end, we investigate the following research questions.

RQ1: How are mock sharing levels (L0–L2) distributed across projects? To understand how developers structure and reuse mock configuration, we first examine the prevalence of

the three mock reuse levels defined in Section III. Projects may rely almost entirely on locally scoped, non-reused mocks (L0), or they may employ shared mock configuration either fully (L1) or partially (L2). Characterizing how these levels are distributed across real-world projects provides essential context for the rest of our investigation: it establishes the baseline frequency of reuse, reveals whether shared mocking is an isolated or systematic practice, and identifies the structural conditions under which reuse tends to appear.

RQ2: How do mock sharing levels change over time within projects? While RQ1 provides a static snapshot, RQ2 examines historical evolution. If developers recognize the maintainability benefits of shared stubbing logic, we might observe transitions from L0 to L1/L2 as projects mature. Conversely, a flat or declining trend may indicate barriers to adopting shared mocking patterns or a lack of incentives to reorganize mocks. This temporal perspective links the descriptive landscape (RQ1) to the potential impacts examined in RQ3.

RQ3: What is the impact on test complexity when mocks are converted between sharedness levels (L0–L2)? RQ3 investigates the complexity impact, using CCTR metric (see Section II-B), of different mock levels by simulating the **Upgrade**

and **Downgrade** transformations defined in Section III-C. The motivation for this conversion-based analysis is to move beyond static correlations and isolate the structural impact of logic relocation. Specifically, **Upgrades** ($L0 \rightarrow L1/L2$) allow us to quantify the latent potential for complexity reduction through extraction, while **Downgrades** ($L1/L2 \rightarrow L0$) reveal the actual “complexity savings” current abstractions provide by exposing the redundancy they conceal. By applying these symmetric shifts, we can determine whether higher-level sharing patterns ($L1/L2$) offer a consistent structural advantage or if the resulting abstractions introduce overhead that offsets their theoretical benefits.

Taken together, the three research questions provide a coherent, multi-perspective investigation of mock sharedness. We aim to understand not only *how* mock sharedness appears in real-world systems, but also *why* it matters, laying essential groundwork for future techniques aimed at improving mock reuse and reducing test maintenance costs.

V. STUDY SUBJECTS

Subject Selection: Our study begins with 260 Apache Software Foundation (ASF) Java projects spanning a wide range of application domains, including distributed systems, databases, messaging middleware, RPC frameworks, configuration management, and developer tooling [30], [47]. Mockito is the most widely adopted mocking framework in the Java ecosystem [8], and its core abstractions (mock creation, shared setups, partial stubbing, and verification patterns) align closely with other mainstream frameworks such as EasyMock [34], JMock [33], and PowerMock. Studying Mockito therefore provides excellent coverage of industry-relevant mocking practice and yields insights that generalize to other mocking abstractions.

Using our AST-based Mockito parser, we extracted all Mockito-created mock objects and their associated stubbing and verification sequences. Among the 260 ASF projects, 112 contained at least one Mockito mock. To ensure reliable pattern estimation, we retained only projects with at least ten mock objects—following established practice in empirical clone and test-pattern studies [48], [49]—resulting in a final dataset of 97 projects. All subsequent analyses are conducted on this set.

Table II summarizes the structural and scale-related characteristics of the 97 selected projects.

Codebase Size. Projects range from 17 KLoC to over 3.2 MLoC, with a median size of 328 KLoC. The number of Java files spans from very small modules to large multi-thousand-file systems. These values confirm that the dataset primarily consists of substantial, mature production codebases. Following common software-repository practice, we classify systems into small (≤ 100 KLoC), medium (100–500 KLoC), and large (≥ 500 KLoC). As shown in Table II, more than 85% of the systems fall into the medium or large categories, indicating that the dataset represents realistic, industrial-scale Java projects.

Test Code Scale. Test suites in these projects are sizable: the number of test cases ranges up to 21,589, and test suites up to 6,570. Mock usage is also extensive, with projects containing

TABLE II
OVERALL STATISTICS OF THE 97 ANALYZED PROJECTS.

Project Size Categories (by KLoC)				
Category	Small	Medium	Large	Total
Definition (KLoC)	≤ 100	100–500	≥ 500	—
# Projects	12	52	33	97
Proportion	12%	54%	34%	100%
Codebase Size				
Metric	Min	Max	Avg	Med
KLoC	17	3,263	564	328
Java files	0	19,154	3,080	1,860
Test Code Scale				
Metric	Min	Max	Avg	Med
Test suites	25	6,570	648	246
Test cases	37	21,589	3,761	1,759
Mock objects	10	10,120	594	112
Project Age				
Metric	Min	Max	Avg	Med
Years	3	26	14	14

an average of 594 mock objects and some exceeding 10,000. This provides a rich environment for analyzing how developers create, reuse, and share mock logic in practice.

Project Age. The projects in our dataset reflect a wide range of maturity levels, with ages spanning from 3 to 26 years and a median age of 14 years (Table II). This distribution indicates that our dataset includes both relatively young ecosystems and long-lived, well-established codebases. Such variation is important because mocking practices evolve as systems accumulate technical debt, grow in complexity, and expand their test suites [50]. Including projects across this maturity spectrum ensures that our analysis captures stable, real-world mocking behavior rather than transient patterns associated with early-stage development.

Thoroughness and Representativeness. The dataset exhibits substantial diversity in domain, maturity, and development scale, while also containing extensive and active test suites that make heavy use of mocking. Mockito’s dominant position in Java testing, combined with its conceptual correspondence to other mocking frameworks, ensures that the behaviors we observe reflect mainstream testing practice rather than tool-specific conventions. Together, these factors make our empirical investigation both comprehensive and broadly representative of real-world software development.

VI. STUDY APPROACH

Our study follows a multi-stage process to obtain the data for analysis. As shown in Figure 1, we first establish two fundamental processing engines—the *Mock Sharedness Level Mapping Pipeline* and the *Level Conversion Simulation*—which obtain the data required to answer our research questions (RQ1, RQ2, and RQ3 introduced in Section IV) using specific metrics and data counting methods.

A. Foundational Processing Engines

The foundational layer of our study consists of two primary mechanisms for data generation:

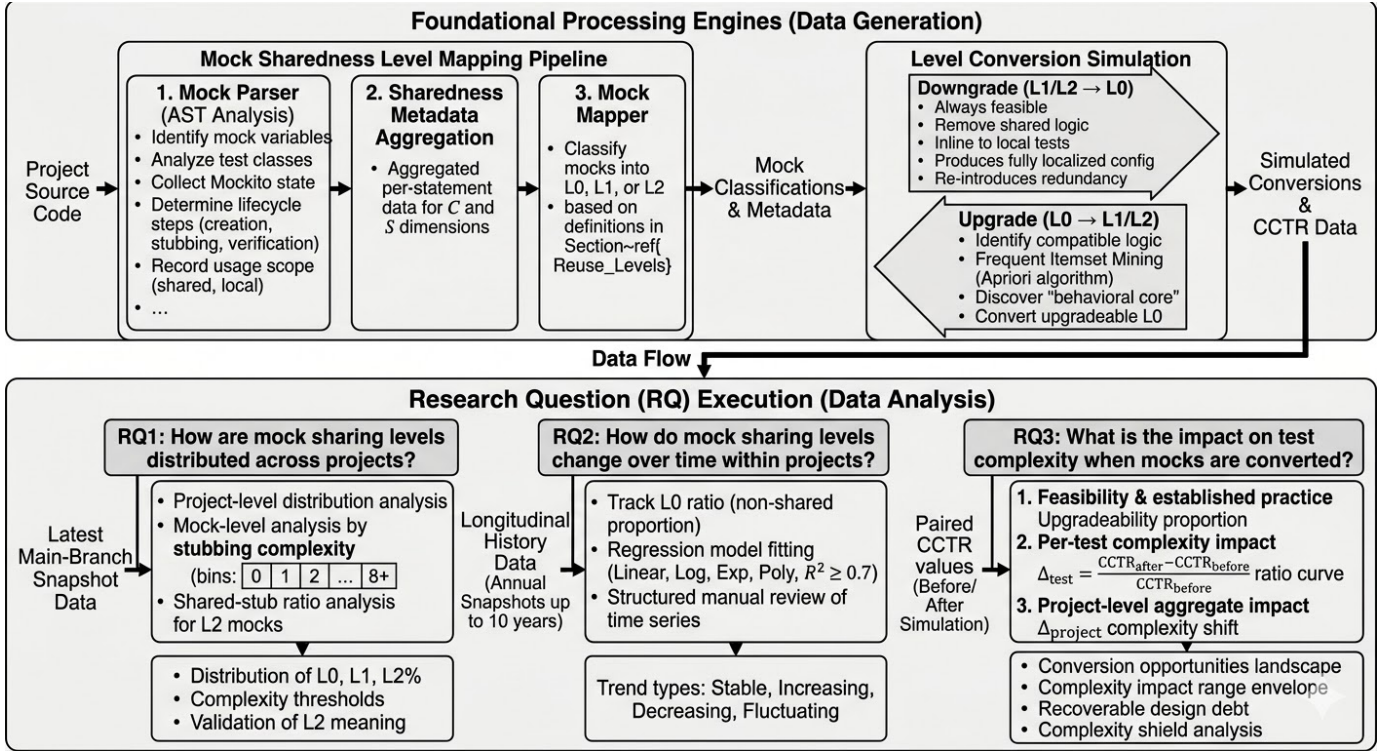


Fig. 1. Overview of approach.

a) *Mock Sharedness Level Mapping Pipeline.*: This pipeline takes as input the source code of each project and produces, for every mock object, its classified reuse level (L0, L1, or L2) along with the underlying per-statement sharedness metadata needed for downstream analysis. First, using the *Mock Parser*, we perform AST-based analysis on each test class. For every mock variable identified via Mockito's creation API (e.g., `mock()`, `@Mock`), we collect all statements that reference that variable and classify each into one of the three lifecycle steps—creation, stubbing, or verification—based on the invoked Mockito API (e.g., `when().thenReturn()` for stubbing, `verify()` for verification). Simultaneously, we record the *usage scope* of each statement: a statement is labeled *shared* if it resides in a shared setup routine (e.g., `@BeforeEach`, `@BeforeAll`) or a helper method callable from multiple tests, and *local* if it resides within a single `@Test` method. When a mock variable is initialized in shared setup and used across multiple test methods, it is treated as a single mock object. Next, the *Mock Mapper* classifies each mock object into L0, L1, or L2, based on the aggregated per-statement sharedness metadata of each lifecycle dimension—*C* and *S*—following the definitions in Section III. This mapping serves as the foundation for all subsequent analyses.

b) *Level Conversion Simulation.*: Taking the mock objects classified in the previous stage, this engine performs structural transformations between non-shared and shared configurations.

We define these transformations as *simulations* because the engine treats conversions structurally rather than syntactically. The primary goal is to efficiently assess the feasibility and

complexity impact of alternative organization strategies without the overhead of ensuring a production-ready, compilable refactoring. While a conversion may be structurally valid in our model, it does not guarantee a direct, compilable substitution due to potential real-world barriers such as cross-package visibility or naming conflicts. This approach allows us to isolate the architectural and cognitive impact of reuse from the linguistic constraints of the Java language.

The engine handles two symmetric directions to measure both the potential gains of sharing and the realized benefits of existing abstractions:

- Downgrade (L1/L2 → L0):** This transformation is always structurally feasible. The engine removes the shared creation and stubbing logic of each mock from its shared location (e.g., a helper method or `@BeforeEach` block) and inlines it directly into every test method that references the mock. This produces a fully localized configuration, which effectively re-introduces the redundant complexity that sharing previously concealed.
- Upgrade (L0 → L1/L2):** Upgrading non-shared mocks requires identifying compatible mocking logic across different test cases. We treat this as a frequent itemset mining problem and apply the **Apriori algorithm** [51]. For each mocked class, every L0 mock instance is treated as a *transaction*, and the methods it stubs are treated as *items*. The algorithm identifies method subsets that recur across multiple objects with a minimum support threshold of $\theta = 2$. This is appropriate here because it provides a principled way to discover the “behavioral core” of a mock that is actually reused in practice. L0 mocks that do not participate in a frequent set are deemed non-

upgradeable. For upgradeable mocks, the engine extracts the common frequent set into a shared helper. If the frequent set covers all stubs of the participating mocks, the result is an L1 configuration; if it covers only a subset, the result is L2, with the remaining unique stubs retained locally.

B. Research Question Execution

Building on the raw data produced by the two foundational engines, we apply specific metrics and analytical methods to answer each research question. The following subsections describe how the pipeline outputs are processed for RQ1–RQ3.

a) *RQ1: How are mock sharing levels (L0–L2) distributed across projects?*: RQ1 operates on the *latest main-branch snapshot* of each project. Using the L0/L1/L2 labels assigned by the mapping pipeline, we conduct three complementary analyses. First, for each of the 97 projects, we compute the percentage of mocks at each reuse level to obtain the *project-level distribution*, which reveals how common shared mocking practices are across the ecosystem. Second, to understand under what structural conditions developers adopt reuse, we perform a *mock-level analysis by stubbing complexity*: for each mock object, we count the number of distinct methods it stubs and group mocks into bins of increasing stub count (0, 1, 2, ..., 8+), then compute the proportion of L0, L1, and L2 mocks within each bin. This analysis identifies behavioral thresholds at which developers shift from local mocking toward shared structures. Third, to validate that L2 represents meaningful partial reuse rather than incidental overlap, we compute the *shared-stub ratio* for every L2 mock—defined as the number of stub statements located in shared code divided by the total number of stub statements for that mock—and examine the distribution of this ratio across all L2 instances.

b) *RQ2: How do mock sharing levels change over time within projects?*: While RQ1 characterizes mock reuse from the latest snapshot, RQ2 examines longitudinal evolution. We trace each project’s history backward from its latest main-branch commit and collect annual snapshots, up to a maximum depth of ten years. Not all projects have a full ten-year history; the number of snapshots per project therefore varies according to its age and the availability of historical commits. For each snapshot, we run the mapping pipeline to classify all mock objects and compute the L0 ratio—the proportion of non-shared mocks. Because L1 and L2 together form the complement of L0, tracking the L0 ratio is equivalent to tracking the overall adoption of shared mocking practices.

Our longitudinal analysis proceeds in two stages. First, for each project we fit multiple candidate regression models—including linear, logarithmic, exponential, and low-degree polynomial curves—to the annual L0 time series. Model selection is based primarily on the coefficient of determination ($R^2 \geq 0.7$), with priority given to simpler functional forms to avoid overfitting. Second, guided by the fitted trendline but not solely determined by it, we perform a structured manual review of each project’s time series to classify its temporal behavior into one of four trend types: *stable* (L0 ratio remains within a narrow band), *increasing* (L0 grows

over time, indicating declining reuse), *decreasing* (L0 declines, indicating growing reuse), or *fluctuating* (L0 varies irregularly without a sustained directional trend).

c) *RQ3: What is the impact on test complexity when mocks are converted between sharedness levels?*: RQ3 uses the paired CCTR values produced by the Level Conversion Simulation engine to quantify how mock sharing strategies affect test maintainability. The analysis proceeds in three stages.

Feasibility and established practice. We first characterize the conversion landscape at the project level. For each project, we compute the proportion of its L0 mocks that are upgradeable to L1/L2 (as determined by the Apriori-based feasibility assessment in the simulation engine), the proportion of L1/L2 mocks that are downgradeable to L0 (always 100%), and the proportion of non-upgradeable L0 mocks. We then examine how upgrade potential is distributed across the 97 projects to determine whether conversion opportunities are concentrated in a few projects or represent a systemic characteristic of the ecosystem.

Per-test complexity impact. For each project, we compute the CCTR change ratio for every test method affected by a conversion:

$$\Delta_{\text{test}} = \frac{\text{CCTR}_{\text{after}} - \text{CCTR}_{\text{before}}}{\text{CCTR}_{\text{before}}}$$

where $\text{CCTR}_{\text{before}}$ and $\text{CCTR}_{\text{after}}$ are the CCTR scores of the test method before and after the simulated conversion. We then rank all affected test cases within each project by the magnitude of Δ_{test} in descending order and construct a per-project curve: at each percentile threshold $X\%$, the curve reports the CCTR change ratio of the test case at that rank, indicating that the top $X\%$ most impactful test cases all experience at least that level of change. Plotting these curves for all 97 projects on a single figure produces an envelope that reveals the range of per-test complexity impact across the ecosystem at any given conversion selectivity (e.g., converting only the top 40% most impactful mocks).

Project-level aggregate impact. To capture the system-level effect, we aggregate across all affected test methods within each project:

$$\Delta_{\text{project}} = \frac{\sum_{t \in T_{\text{affected}}} \text{CCTR}_{\text{after}}(t) - \sum_{t \in T_{\text{affected}}} \text{CCTR}_{\text{before}}(t)}{\sum_{t \in T_{\text{affected}}} \text{CCTR}_{\text{before}}(t)}$$

where T_{affected} is the set of all test methods affected by the conversion of all eligible mocks in the project. This metric measures the total complexity shift at the project level—representing either the “design debt” recoverable through upgrading or the “complexity shield” currently provided by existing shared configurations. We compute Δ_{project} separately for upgrades and downgrades and examine their distributions across projects.

VII. RESULTS

A. RQ1: Mock Reuse Level Distribution

Finding 1: Project-Level Mock Reuse Distribution. Across the 97 eligible projects, we observe a strong ecosystem-

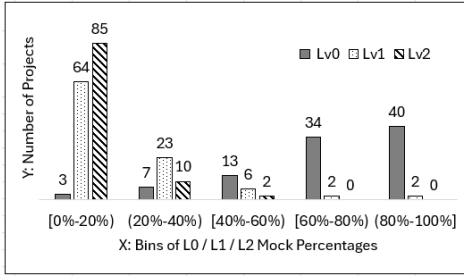


Fig. 2. Histogram of L0/L1/L2 Mock Percentages Across Projects

level skew toward non-reuse: **most projects rely predominantly on L0 mocks**. As shown in Figure 2, 74 projects place 60–100% of their mocks in L0, indicating that mock configuration is overwhelmingly defined within individual test methods rather than shared. L1 (fully shared mocks) exhibits a modest but scattered presence: 13 projects fall in the 0–20% range, 35 projects in 20–40%, 23 in 40–60%, and only a handful exceed 60%. This suggests that while some projects adopt shared mock setup methods, this practice rarely becomes dominant. L2 (partially shared mocks) is the least prevalent structure at the project level. A striking 85 out of 97 projects have fewer than 20% L2 mocks, and all remaining bins contain at most ten projects. These low proportions indicate that hybrid reuse—shared configuration with test-specific customization—is uncommon when viewed at the project scale. Taken together, these project-level results reveal that **mock reuse is not broadly adopted across the ecosystem**, setting the stage for examining whether reuse emerges under specific structural conditions.

Finding 2: Mock-Level Variation by Stubbing Complexity. To understand *when* developers adopt reuse, we analyze reuse levels at the mock level with respect to stubbing complexity. Figure 3 shows the proportion of L0, L1, and L2 mocks across increasing stub-count bins. For mocks with zero or one stub, L0 overwhelmingly dominates, with only small fractions of L1 and L2. As stubbing complexity increases, a contrasting pattern emerges: **L0 usage steadily declines**, while **L2 usage rises sharply**, becoming the dominant structure for mocks with five or more stubs. L1 remains consistently low across all complexity levels. This pattern indicates that **mock reuse occurs primarily when the mocking task becomes nontrivial**. When mocks involve multiple stubbing statements, developers shift from purely local configuration (L0) toward *partial reuse* (L2), reflecting a behavioral mechanism in which the cost of duplication motivates shared configuration.

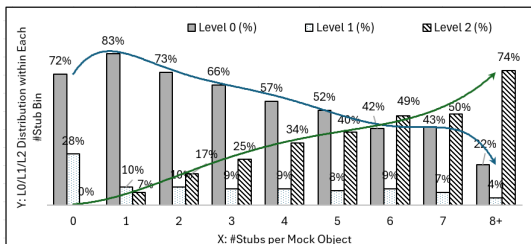


Fig. 3. Trends of L0/L1/L2 Usage by Number of Stubs

Finding 3: Shared-Stub Ratios Validate the Meaning of L2. To ensure that L2 represents meaningful partial reuse rather than accidental overlap, we analyze the *shared-stub ratio* for all L2 mocks—the proportion of their stubbing logic that occurs in shared setup or helper methods. As shown in Figure 4, the shared-stub ratio distribution for all L2 mocks is strongly skewed toward high reuse. L2 mocks appear across all ratio bins, but the largest concentration occurs in the 80–100% range (2,650 mocks), indicating that most L2 mocks reuse the majority of their stubbing logic. Moderate reuse levels (20–60%) are also common, with approximately 1,100–1,200 mocks in each bin, whereas low-reuse cases (0–20% and 60–80%) are substantially smaller. This distribution confirms that L2 mocks generally embody substantial shared configuration rather than incidental overlap. **These results confirm that L2 mocks embody substantial shared configuration, supporting the interpretation that L2 captures intentional, structured reuse of mock logic paired with localized customization.** This completes the RQ1 analysis by validating that the L2 category reflects meaningful reuse behavior rather than structural noise.

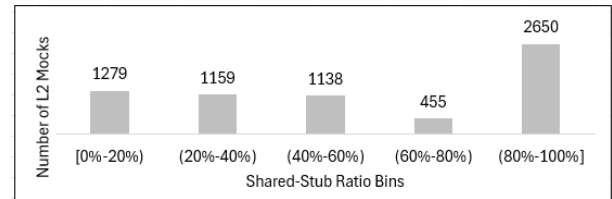


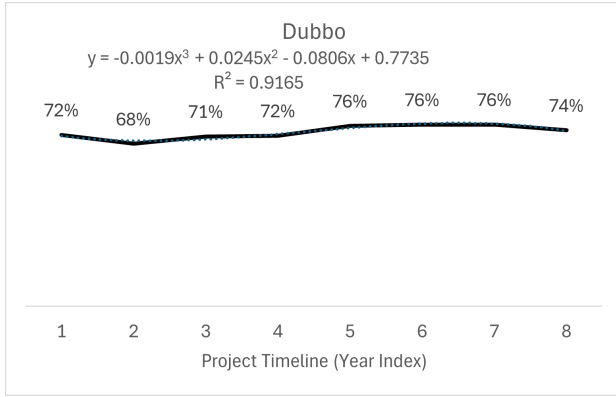
Fig. 4. Shared-Stub-Ratio Distribution Across L2 Mocks

RQ1 Summary & Implications

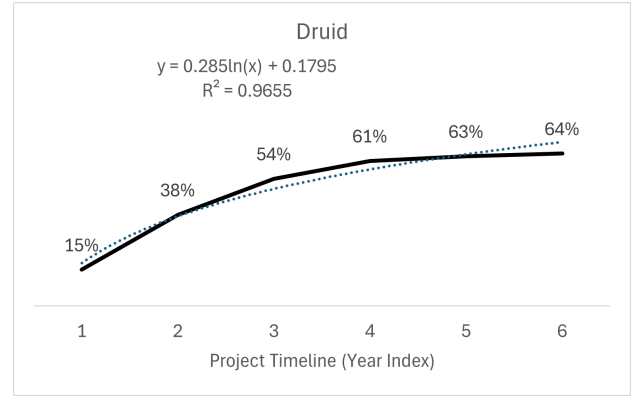
Mock reuse is uncommon across projects: most systems rely heavily on locally defined mocks (L0), with L1 and especially L2 appearing only in small proportions. Yet when mocking becomes complex, developers reliably shift toward partial reuse (L2), indicating that reuse emerges as a response to increasing stubbing effort. Analysis of shared-stub ratios confirms that L2 reflects substantial shared configuration rather than incidental overlap. Together, these findings reveal a structural gap: developers value reuse under complexity, but projects rarely adopt reuse patterns systematically.

B. RQ2: Mock Share Levels Longitudinal Evolution

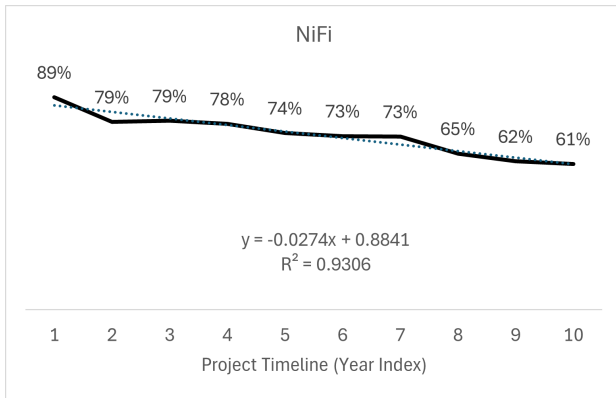
The evolution of L0 ratios across the 97 projects falls into four distinct temporal patterns, distributed roughly evenly: 37 projects (38%) exhibit stable trends, 22 (23%) show increasing trends, 21 (22%) display decreasing trends, and 17 (17%) demonstrate fluctuating trends. A *stable* trend indicates that a project’s reliance on unshared mocks persists with little deviation, reflecting long-standing consistency in mocking practices. An *increasing* trend captures projects where L0 grows over time, suggesting that shared mocks fail to keep pace with



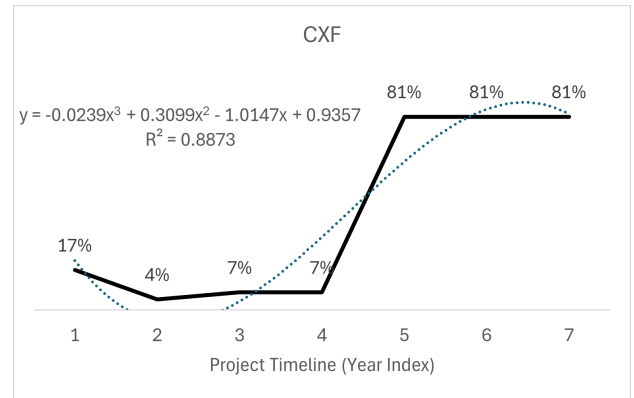
(a) Stable trend (Apache Dubbo)



(b) Increasing trend (Apache Druid)



(c) Decreasing trend (Apache NiFi)



(d) Fluctuating trend (Apache CXF)

Fig. 5. Representative trend patterns of mock sharing evolution across projects (RQ2).

expanding test suites. A *decreasing* trend reflects projects in which shared mocks grow faster than unshared ones, pointing to emerging reuse-oriented behavior. Finally, a *fluctuating* trend denotes systems whose L0 ratios vary irregularly due to small-sample volatility, framework transitions, or architectural shifts. These patterns highlight how rarely projects undergo sustained movement toward greater reuse, despite localized refactoring efforts.

Stable Trends. Across the 37 stable projects, we observe three recurring forms of stability.

- *Strict Stability*, where both the number of mocks and the L0 ratio remain nearly constant across releases, reflecting a project that neither expands nor revises its mocking strategy.
- *Narrow-band Stability*, which is most common, and exemplified by Dubbo in Figure 5a. The absolute number of mocks changes dramatically, but the L0 ratio fluctuates only within a small interval (72%–76%); this indicates that increases in shared mocks (L1/L2) and unshared mocks (L0) occur at similar rates.
- *Degenerate Stability* pattern, where nearly all mocks remain unshared (L0 $\approx$ 100%), producing a flat trajectory regardless of project evolution. These scenarios show that stability reflects not the absence of change in mocking activity, but the persistence of a consistent structural

pattern: projects tend to retain their original reliance on local mocking, and expansions in test suites rarely shift the balance toward greater reuse.

Increasing Trends. The Druid project in Figure 5b exemplifies this pattern: its L0 ratio rises from 15% to over 60% across six annual snapshots, following a logarithmic trajectory in which the rate of increase slows as the project matures. This behavior shows that as new tests are added or existing tests are revised, developers introduce unshared mocks at a faster rate than shared ones, causing reuse to decline over time. Among the 22 projects that exhibit an increasing trend, we observe two underlying mechanisms:

- *Redundancy-Driven Expansion*, where the total number of mocks steadily grows, but the growth is dominated by L0 rather than L1/L2, meaning that developers continue to duplicate mock setup logic even as the test suite expands.
- *Abstraction Erosion*, where the total number of mocks decreases—often due to file deletions or architectural refactoring—but the reductions disproportionately impact shared mocks, which raises the L0 ratio even when absolute L0 counts remain stable.

Both mechanisms reveal that once projects begin leaning heavily on unshared mocks, they tend to reinforce this pattern, making reuse less likely to emerge naturally as the codebase

evolves.

Decreasing Trends. The NiFi project, shown in Figure 5c, exemplifies a pattern we define as *Shared-First Accretion*. In this scenario, the L0 ratio drops from 89% to 61% across ten yearly snapshots, with a clear downward trajectory captured by a linear fit. This behavior indicates that as the project evolves, developers introduce shared mocks (L1/L2) at a faster rate than unshared ones, resulting in a gradual increase in reused configuration. All 21 projects exhibiting a decreasing trend share this same underlying mechanism: while the test suite grows over time, the growth is disproportionately concentrated in shared mocks rather than in L0. In practical terms, *Shared-First Accretion* reflects situations where developers intentionally consolidate mock logic into shared setup methods or design new tests to inhabit existing shared infrastructure rather than duplicating mock definitions. Although these cases demonstrate that reuse-oriented behavior is achievable, they remain the exception rather than the norm. This highlights that systematic consolidation is a deliberate architectural choice, typically emerging only when projects actively reinforce their shared testing utility.

Fluctuating Trends. The CXF project, shown in Figure 5d, is a representative example: its L0 proportion initially drops from 17% to 4%, remains low for several snapshots, and then rises sharply to 81%. This abrupt transition reflects a structural shift in the project—likely associated with framework migration or architectural reorganization—that disrupts earlier mocking patterns and produces a non-monotonic evolution curve. Among the 17 fluctuating projects, we observe three recurring sources of instability:

- **Small-sample Volatility:** projects with a very limited number of mocks (often fewer than 20) experience large swings in the L0 ratio because even minor additions or deletions disproportionately affect the percentage.
- **Major Project-Level Transitions,** as in CXF, where changes such as switching from JUnit 4 to JUnit 5 correlate with abrupt shifts in mocking practices.
- **Early-Phase Adoption Effects:** as mocking is introduced gradually into a project, the L0 ratio may oscillate—sometimes rising, sometimes falling—before stabilizing or transitioning into a more predictable trend type.

These cases underscore the potential sensitivity of L0 to both project scale and testing infrastructure transitions.

RQ2 Summary: The Mechanics of Mock Evolution

Mock sharing evolution is governed by four primary forces rather than random change:

- **Structural Inertia (Stable - 38%):** Most projects exhibit a “path dependency” where the initial choice of mocking style (usually L0) persists regardless of suite growth. This suggests that without external intervention, the architectural “gravity” of a project keeps reuse levels constant.
- **The Redundancy Drift (Increasing - 23%):** Driven by *Redundancy-Driven Expansion* or *Abstraction Erosion*, these projects actively lose

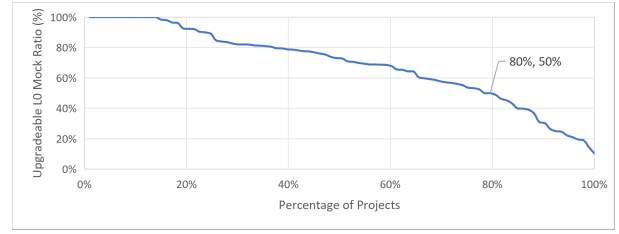


Fig. 6. Project-Level Distribution of Upgradeable L0 Mocks.

reuse over time. This indicates that as complexity increases, developers favor the “path of least resistance” (copy-pasting mocks) over maintaining shared abstractions.

- **Intentional Consolidation (Decreasing - 22%):** *Shared-First Accretion* is the only pattern showing improved health. However, its rarity suggests that increasing reuse is not a natural byproduct of project maturity, but a result of deliberate, disciplined architectural enforcement.
- **Migration Shock (Fluctuating - 17%):** Abrupt shifts are rarely organic; they are usually “shocks” caused by framework transitions (e.g., JUnit 4 to 5). This reveals that mocking patterns are deeply coupled to the underlying testing infrastructure.

Implication: Mocking debt is “sticky”—once L0-dominant patterns take root, they are rarely reversed unless the project undergoes a major infrastructural overhaul.

C. RQ3: Impact of Mock Reuse Level Conversion

a) **Finding 1: Established Practice and Upgrade Potential.** Across the 97 projects, 16,399 mocks (29%) are already shared (L1/L2). Significantly, although 100% of these instances are structurally downgradeable to L0 (i.e., the shared logic could be inlined in each test case), they are constructed as shared by the developers. This represents a clear established best practice, providing strong evidence that developers are aware of mock reuse benefits and have opted for shared configurations to manage complexity.

Building on this awareness, 31,874 mocks (56%) are currently non-shared L0 instances that are potentially upgradeable to L1/L2. Only a small minority (15%) are non-upgradeable. This lack of upgradeability typically stems from a lack of stubbing commonality; these mocks contain highly specialized behavior or unique return values that are not replicated across other test cases. The fact that the majority of the mocks in the ecosystem are upgradeable suggests a significant opportunity for systematic mock sharing improvement. Figure 6 shows that 80% of projects have an improvement potential exceeding 50% of their mocks, indicating that even in projects where sharing is already practiced, there is substantial room to further apply these established patterns to reduce test complexity.

b) **Finding 2: Impact on test case complexity.** Figure 7a illustrates the potential for complexity reduction per impacted test case. The “envelope” represents the distribution

of results across all 97 analyzed projects, showing the upper and lower bounds of CCTR decrease per test case. At the 40% threshold—where only the most impactful L0 mocks are upgraded—affected tests across the ecosystem experience a per-test CCTR reduction ranging from 14% to 67% across the 97 projects. This wide band indicates that while the magnitude of improvement varies by project, the benefit of moving toward shared configurations is universally positive and often substantial. Figure 7b provides the mirroring perspective: the impact of abandoning shared configurations. By downgrading the top 40% of shared mocks, the complexity per test case increases significantly, with the project envelope showing a penalty between 24% and 160%. The fact that even the lower bound (24%) represents a notable spike in complexity confirms that the existing shared mocks—the established best practices—are currently providing a vital “complexity shield” for these test suites.

c) **Finding 3: Project-level aggregate impact.**: Figure 8a shows the distribution of untapped improvement potential across projects. Among the 93 projects with upgradeable mocks, the histogram reveals a strong concentration of benefit: 46 projects achieve a total CCTR per project reduction between 20–40%, and another 22 projects achieve 40–60%. Overall, 96% of projects (89/93) would reduce the aggregate complexity of their affected tests by at least 20%. This concentration in the 20–60% range demonstrates that the architectural gain of sharing is not limited to a few specific tests but scales significantly at the project level. Figure 8b quantifies the lost benefit if established sharing practices were abandoned. Uniformly downgrading shared mocks to non-shared configurations results in a widespread complexity spike. The histogram shows that for 42 projects, total CCTR per project for affected tests would increase by 20–40%, while 25 projects would see a 40–60% increase. Notably, a “high-penalty” tail exists where 14 projects would see complexity increase by 80% or more—with 7 of those projects seeing their complexity more than double (>100% increase). These results confirm that existing shared mocks currently serve as a critical architectural defense, preventing a substantial inflation of test suite complexity.

RQ3 Summary: The Economics of Mock Sharing

Our simulation of sharing-level transitions reveals that mock configuration is a high-leverage architectural lever:

- **The “Complexity Shield” (Current State):** The 29% of mocks already shared by developers are not accidental; they serve as a critical defense against suite bloat. Downgrading them would trigger a “complexity spike” (up to 160% per test), confirming that existing shared logic is a deliberate, expert-driven barrier against unmanageable test code.
- **Untapped Design Debt (Future Potential):** A massive 56% “improvement gap” exists where L0 mocks could be upgraded but aren’t. This

suggests that while developers recognize the value of sharing, they lack the analytical visibility to identify these opportunities manually as the project scales.

- **High-Efficiency Refactoring:** Impact is non-linear. By targeting only the “Top 40%” of impactful mocks for conversion, projects can capture the bulk of the benefit—reducing test complexity by up to 67%. This indicates that systematic sharing is a low-effort, high-reward strategy for reducing maintenance overhead.

Implication: Modern test suites suffer from “latent redundancy.” Automated identification of the behavioral core of mocks can recover significant project-level maintainability (20–60% CCTR reduction) that is currently obscured by local, unshared definitions.

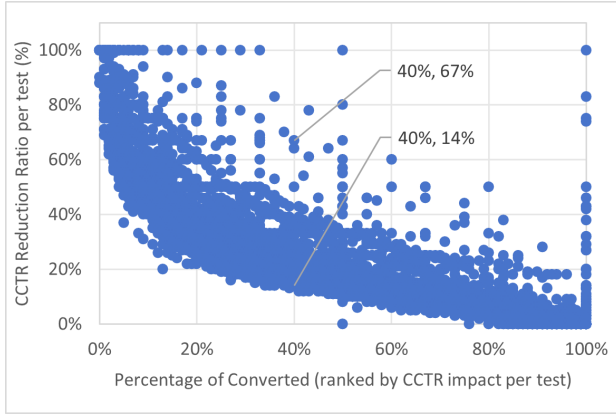
VIII. RELATED WORK

A. Empirical Studies of Mocking Practices

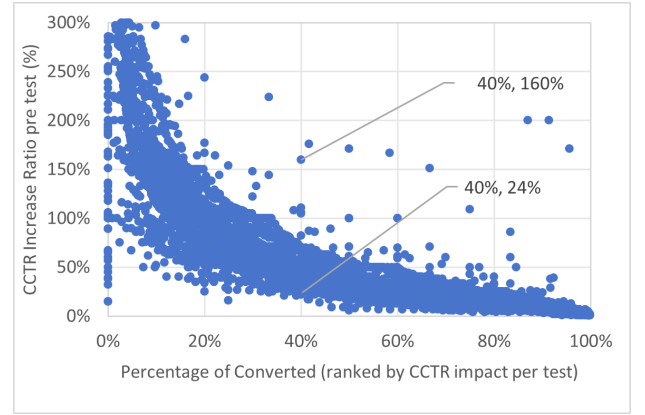
The concept of mock objects was formalized by Freeman et al. [5], who introduced jMock to enable isolated unit testing through programmable substitutes for real dependencies. EasyMock [34], introduced around the same time, further popularized framework-based mocking in Java. Since then, mocking has been applied across diverse domains: embedded systems [36], [37], cloud platforms [38], and mobile applications [39], [40]. Wang et al. [6], [52] proposed automatic refactoring frameworks that replace inheritance-based test structures with Mockito-based alternatives, highlighting developers’ increasing preference for framework-driven mocking.

Empirical studies have characterized mocking adoption at scale. Mostafa and Wang [9] conducted an early study confirming widespread mock adoption in open-source Java projects. Spadini et al. [7], [10] investigated mocking decisions and their relationship with code characteristics, finding that mocking practices correlate with coupling and complexity. Xiao et al. [8] presented the first large-scale study of mock framework usage in Apache projects, identifying Mockito as the dominant framework and observing that a small set of high-frequency APIs accounts for 78%–100% of usage—a concentration that implies a high degree of similarity in mock creation logic across test cases. Zhu et al. [29] proposed MockSniffer to recommend mocking decisions based on project context. More broadly, Zaidman et al. [11] and Beller et al. [12] studied the co-evolution of test and production code, establishing that test maintenance constitutes a significant and ongoing cost in software development—a finding that motivates the need to understand how specific test constructs, such as mocking logic, are organized over time.

Zhao [20] conducted the first empirical investigation of mock clones across six open-source Java projects, introducing the concept of mock clones as repeated mocking sequences and demonstrating that 71%–100% of frequently mocked classes contain such duplication. Building on this foundation, Zhao [21] developed an automated framework for detecting

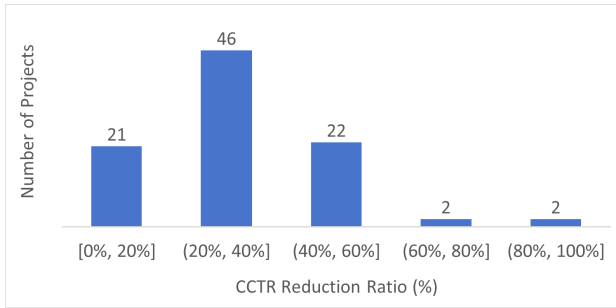


(a) Upgrading L0 mocks (L0→L1/L2)

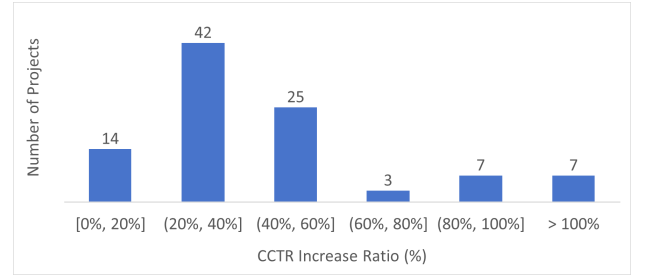


(b) Downgrading shared mocks (L1/L2→L0)

Fig. 7. Impact of mock sharing level conversion on individual test complexity (CCTR).



(a) Upgrading all L0 mocks



(b) Downgrading all shared mocks

Fig. 8. Project-level aggregate impact of mock sharing level conversion on test complexity (CCTR).

and refactoring mock clones using a combination of AST-based analysis and LLMs, with manual validation confirming that 21 of 24 submitted refactoring pull requests were accepted by developers. While these studies establish that mock duplication is prevalent, harmful, and amenable to refactoring, they do not systematically examine *how* mocking logic is organized and shared across test cases—the central focus of our work.

B. Test Code Maintainability and Complexity

Test code quality has been studied extensively through the lens of test smells—structural patterns that indicate potential maintainability problems [13], [16]. Van Rompaey et al. [53] proposed metrics-based detection for General Fixture and Eager Test smells, while Greiler et al. [54] automated the detection of test fixture strategies and associated smells. Bavota et al. [14], [18] found that test smells negatively impact software maintenance, and Spadini et al. [17] established correlations between test smells and production code quality. Longitudinal studies further show that such smells tend to persist and accumulate rather than being resolved organically [15], [44].

The AAA (Arrange-Act-Assert) pattern provides a structural framework for understanding test organization [46]. Wei et al. [55], [24] developed automated approaches to identify AAA segments and empirically analyzed how developers structure test cases, finding that the Arrange phase is often the most complex and variable portion of a test. The duplication of setup

logic within this Arrange phase has been specifically recognized as a maintainability concern. Meszaros [4] catalogued fixture setup strategies—including inline, implicit, and shared setup—and characterized their tradeoffs. Greiler et al. [22] empirically showed that fixture setup choices directly shape the emergence of fixture-related smells over time. Alves et al. [23] further proposed fixture-sharing models to reduce setup duplication across test classes. However, these studies address general fixture duplication and do not examine the specific structure or reuse patterns of mocking logic within the Arrange phase. While clone refactoring techniques [56], [42] provide general strategies for eliminating code duplication, they are not tailored to the domain-specific semantics of mock lifecycle operations.

Ouédraogo et al. [26] proposed CCTR (Cognitive Complexity for Test Readability), a readability-aware metric that measures test complexity by accounting for test-specific structures such as setup logic and assertions. We adopt CCTR in our study to quantify the complexity implications of different mock sharing strategies. Unlike prior work that treats test complexity holistically, our study isolates the contribution of mocking logic organization to overall test complexity.

Despite this progress, no prior work has systematically studied *mock reuse patterns*—the ways developers organize and share mocking logic across test cases—or quantified their impact on test complexity. Our study fills this gap by defining a taxonomy of mock sharing levels (L0–L2), analyzing their

distribution and evolution across 97 Apache projects, and establishing their correlation with test complexity as measured by CCTR.

IX. THREATS TO VALIDITY

Our study has several limitations that should be considered when interpreting the results.

Dataset Scope. We analyze only Apache Software Foundation projects, which may not represent all Java development practices. However, ASF projects span diverse domains, scales (17K–3.2M LoC), and development contexts, making them a reasonable proxy for mature, production-quality systems. Furthermore, our analysis focuses exclusively on Mockito-based mocks, excluding other frameworks such as EasyMock [34], PowerMock [57], and JMockit [58]. Prior work has shown that Mockito is the most widely adopted mocking framework in Apache projects [8], so our findings capture the predominant mocking practice in this ecosystem, though they may not generalize to projects using alternative frameworks.

Detection Approach. Our feasibility analysis employs the Apriori algorithm to identify mock objects with recurring mocking logic, but does not apply optimization techniques (e.g., greedy or ILP-based methods) to maximize conversion opportunities. As a result, mocks are grouped based on shared stubbing patterns rather than optimal clustering. This may underestimate the true potential for mock consolidation in some projects.

Conversion Simulation. Our complexity impact analysis simulates mock conversions by programmatically assembling transformed code rather than performing actual refactoring. This approach does not account for real-world constraints such as cross-package visibility, naming conflicts, multiple interdependent mocks within a single test, or other syntactic barriers that may prevent certain conversions in practice. Consequently, some conversions deemed feasible in our analysis may not be directly applicable. However, since CCTR measures structural complexity independent of compilation validity [26], the observed complexity differences remain meaningful indicators of the relative maintainability impact of different mock organization strategies.

Construct Validity. Our complexity analysis relies on CCTR [26] as a proxy for test maintainability. While CCTR has been validated against developer-perceived comprehension effort across 15,750 test suites, it remains a single metric and may not capture all dimensions of test quality. Complementary metrics such as Lines of Code or Cyclomatic Complexity [27] were considered but excluded due to their inability to differentiate between tests with different mock organization strategies, as discussed in Section II-B.

X. CONCLUSION

This paper presents the first large-scale empirical study of mock sharing practices in Java test code, analyzing 56,502 mock objects across 97 Apache projects. We introduce a three-level taxonomy of mock reuse—L0 (non-shared), L1 (fully shared), and L2 (partially shared)—and investigate how these

sharing patterns are distributed, evolve over time, and impact test complexity as measured by CCTR.

Our findings reveal that mock reuse is uncommon at the project level, with 74 out of 97 projects placing 60–100% of their mocks in L0. However, reuse is not absent: when mocking tasks become structurally complex (five or more stubs), developers reliably shift toward partial reuse (L2), which becomes the dominant pattern at high stubbing complexity. Longitudinal analysis across up to ten annual snapshots shows that sharing practices tend to remain stable over project lifetimes, with systematic adoption of shared patterns occurring in only 22% of projects. Through conversion-based simulation, we demonstrate that 56% of currently non-shared mocks are structurally upgradeable to shared configurations. At the project level, upgrading all eligible mocks reduces aggregate test complexity by 20–60% for 96% of projects, while abandoning existing shared configurations would increase complexity by 20–60% or more—with 14 projects seeing increases exceeding 80%. These results provide concrete, quantitative evidence that mock sharing strategy directly impacts test maintainability and that substantial untapped opportunities for complexity reduction exist across the ecosystem.

Our work lays the empirical foundation for future research on automated mock refactoring tools that can identify high-impact sharing opportunities and guide developers toward more maintainable test architectures.

REFERENCES

- [1] P. Runeson, “A survey of unit testing practices,” *IEEE software*, vol. 23, no. 4, pp. 22–29, 2006.
- [2] E. Daka and G. Fraser, “A survey on unit testing practices and problems,” in *2014 IEEE 25th International Symposium on Software Reliability Engineering*. IEEE, 2014, pp. 201–211.
- [3] G. J. Myers, T. Badgett, T. M. Thomas, and C. Sandler, *The art of software testing*. Wiley Online Library, 2004, vol. 2.
- [4] G. Meszaros, *xUnit test patterns: Refactoring test code*. Pearson Education, 2007.
- [5] S. Freeman, T. Mackinnon, N. Pryce, and J. Walnes, “Mock roles, not objects,” in *Companion to the 19th annual ACM SIGPLAN conference on Object-oriented programming systems, languages, and applications*, 2004, pp. 236–246.
- [6] X. Wang, L. Xiao, T. Yu, A. Woepse, and S. Wong, “An automatic refactoring framework for replacing test-production inheritance by mocking mechanism,” in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 540–552.
- [7] D. Spadini, M. Aniche, M. Bruntink, and A. Bacchelli, “To mock or not to mock? an empirical study on mocking practices,” in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 402–412.
- [8] L. Xiao, G. Zhao, X. Wang, K. Li, E. Lim, C. Wei, T. Yu, and X. Wang, “An empirical study on the usage of mocking frameworks in apache software foundation,” *Empirical Software Engineering*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267232273>
- [9] S. Mostafa and X. Wang, “An empirical study on the usage of mocking frameworks in software testing,” in *2014 14th international conference on quality software*. IEEE, 2014, pp. 127–132.
- [10] D. Spadini, M. Aniche, M. Bruntink, and A. Bacchelli, “Mock objects for testing java systems,” *Empirical Software Engineering*, vol. 24, no. 3, pp. 1461–1498, 2019.
- [11] A. Zaidman, B. Van Rompaey, A. van Deursen, and S. Demeyer, “Studying the co-evolution of production and test code in open source and industrial developer test processes through repository mining,” *Empirical Software Engineering*, vol. 16, no. 3, pp. 325–364, 2011.
- [12] M. Beller, G. Gousios, A. Panichella, S. Proksch, S. Amann, and A. Zaidman, “Developer testing in the ide: Patterns, beliefs, and behavior,” *IEEE Transactions on Software Engineering*, vol. 45, no. 3, pp. 261–284, 2017.

- [13] A. Van Deursen, L. Moonen, A. Van Den Bergh, and G. Kok, "Refactoring test code," in *Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. Citeseer, 2001, pp. 92–95.
- [14] G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley, "An empirical analysis of the distribution of unit test smells and their impact on software maintenance," in *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 2012, pp. 56–65.
- [15] M. Tufano, F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, and D. Poshyvanyk, "An empirical investigation into the nature of test smells," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, 2016, pp. 4–15.
- [16] V. Garousi, B. Kucuk, and M. Felderer, "What we know about smells in software test code," *IEEE Software*, vol. 36, no. 3, pp. 61–73, 2018.
- [17] D. Spadini, F. Palomba, A. Zaidman, M. Bruntink, and A. Bacchelli, "On the relation of test smells to software code quality," in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2018, pp. 1–12.
- [18] G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley, "Are test smells really harmful? an empirical study," *Empirical Software Engineering*, vol. 20, no. 4, pp. 1052–1094, 2015.
- [19] M. Greiler, A. Van Deursen, and M.-A. Storey, "Automated detection of test fixture strategies and smells," in *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*. IEEE, 2013, pp. 322–331.
- [20] G. Zhao, L. Xiao, H. Qin, E. A. AlOmar, and S. Wong, "Mock clones in the wild: An empirical investigation across six open-source projects," in *Proceedings of the Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2025.
- [21] G. Zhao, "Automated detection and refactoring of mock clones in Java projects," in *Proceedings of the 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2025, pp. 112–116.
- [22] M. Greiler, A. Zaidman, A. van Deursen, and M.-A. Storey, "Strategies for avoiding test fixture smells during software evolution," in *2013 10th Working Conference on Mining Software Repositories (MSR)*. IEEE, 2013, pp. 387–396.
- [23] E. J. Alves, F. F. Silveira, and F. F. Silveira, "Reuse of fixture setup between test classes," in *Proceedings of the 29th International Conference on Software Engineering and Knowledge Engineering (SEKE)*, 2017, pp. 181–186.
- [24] C. Wei, L. Xiao, T. Yu, S. Wong, and A. Clune, "How do developers structure unit test cases? an empirical analysis of the aaa pattern in open source projects," *IEEE Transactions on Software Engineering*, vol. 51, no. 4, pp. 1007–1038, 2025.
- [25] C. Wei, L. Xiao, T. Yu, X. Chen, X. Wang, S. Wong, and A. Clune, "Automatically tagging the 'aaa' pattern in unit test cases using machine learning models," *IEEE Transactions on Software Engineering*, vol. 49, no. 5, pp. 3305–3324, 2023.
- [26] W. C. Ouédraogo, Y. Li, X. Dang, X. Zhou, A. Koyuncu, J. Klein, D. Lo, and T. F. Bissyandé, "Rethinking cognitive complexity for unit tests: Toward a readability-aware metric grounded in developer perception," in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 797–802.
- [27] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [28] G. A. Campbell, "Cognitive complexity: A new way of measuring understandability," <https://www.sonarsource.com/docs/CognitiveComplexity.pdf>, 2023, version 1.7.
- [29] H. Zhu, L. Wei, M. Wen, Y. Liu, S.-C. Cheung, Q. Sheng, and C. Zhou, "Mocksniffer: Characterizing and recommending mocking decisions for unit tests," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 436–447.
- [30] A. Mockus, R. T. Fielding, and J. D. Herbsleb, "Two case studies of open source software development: Apache and mozilla," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 11, no. 3, pp. 309–346, 2002.
- [31] P. Ammann and J. Offutt, *Introduction to software testing*. Cambridge University Press, 2016.
- [32] K. Beck, *Test-driven development: by example*. Addison-Wesley Professional, 2003.
- [33] S. Freeman, T. Mackinnon, N. Pryce, and J. Walnes, "jmock: supporting responsibility-based design with mock objects," in *Companion to the 19th annual ACM SIGPLAN conference on Object-oriented programming systems, languages, and applications*, 2004, pp. 4–5.
- [34] T. Freese, "Easymock: dynamic mock objects for junit," in *Proceedings of the 3rd International Conference on Extreme Programming and Agile Processes in Software Engineering*. Citeseer, 2002, pp. 1–5.
- [35] "Mockito framework site — site.mockito.org," <https://site.mockito.org>, [Accessed 22-03-2024].
- [36] M. Karlesky, G. Williams, W. Bereza, and M. Fletcher, "Mocking the embedded world: Test-driven development, continuous integration, and design patterns," in *Proc. Emb. Systems Conf. CA, USA*, 2007, pp. 1518–1532.
- [37] S. S. Kim, "Mocking embedded hardware for software validation," Ph.D. dissertation, 2016.
- [38] S. Svensgård and J. Henriksson, "Mocking saas cloud for testing," 2017.
- [39] M. Fazzini, A. Gorla, and A. Orso, "A framework for automated test mocking of mobile apps," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 1204–1208.
- [40] M. Fazzini, C. Choi, J. M. Copia, G. Lee, Y. Takehi, A. Gorla, and A. Orso, "Use of test doubles in android testing: An in-depth investigation," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2266–2278.
- [41] A. Arcuri, G. Fraser, and R. Just, "Private api access and functional mocking in automated unit test generation," in *2017 IEEE international conference on software testing, verification and validation (ICST)*. IEEE, 2017, pp. 126–137.
- [42] M. Fowler, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 2018.
- [43] D. Coleman, D. Ash, B. Lowther, and P. Oman, "Using metrics to evaluate software system maintainability," *Computer*, vol. 27, no. 8, pp. 44–49, 1994.
- [44] F. Palomba, D. Di Nucci, A. Panichella, R. Oliveto, and A. De Lucia, "On the diffusion of test smells in automatically generated test code: An empirical study," in *2016 IEEE/ACM 9th International Workshop on Search-Based Software Testing (SBST)*. IEEE, 2016, pp. 5–14.
- [45] F. Henderson, "Software engineering at google," *arXiv preprint arXiv:1702.01715*, 2017.
- [46] J. Grigg, <http://wiki.c2.com/?ArrangeActAssert/>, 2012.
- [47] P. C. Rigby, D. M. German, and M.-A. Storey, "Open source software peer review practices: a case study of the apache server," in *Proceedings of the 30th international conference on Software engineering*, 2008, pp. 541–550.
- [48] M. Kim, V. Sazawal, D. Notkin, and G. Murphy, "An empirical study of code clone genealogies," in *Proceedings of the 10th European software engineering conference held jointly with 13th ACM SIGSOFT international symposium on Foundations of software engineering*, 2005, pp. 187–196.
- [49] S. Bazrafshan and R. Koschke, "An empirical study of clone removals," in *2013 IEEE International Conference on Software Maintenance*. IEEE, 2013, pp. 50–59.
- [50] T. Mens and T. Tourwé, "A survey of software refactoring," *IEEE Transactions on software engineering*, vol. 30, no. 2, pp. 126–139, 2004.
- [51] F. Bodon, "A fast apriori implementation," in *FIMI*, vol. 3, 2003, p. 63.
- [52] X. Wang, L. Xiao, T. Yu, A. Woepse, and S. Wong, "Jmock: refactoring test-production inheritance by mockito," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 125–129.
- [53] B. Van Rompaey, B. Du Bois, S. Demeyer, and M. Rieger, "On the detection of test smells: A metrics-based approach for general fixture and eager test," *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 800–817, 2007.
- [54] M. Greiler, A. van Deursen, and M. Storey, "Automated detection of test fixture strategies and smells," in *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*, 2013, pp. 322–331.
- [55] C. Wei, L. Xiao, T. Yu, X. Chen, X. Wang, S. Wong, and A. Clune, "Automatically tagging the 'aaa' pattern in unit test cases using machine learning models," *IEEE Transactions on Software Engineering*, 2023.
- [56] N. Tsantalis, D. Mazinanian, and S. Rostami, "Clone refactoring with lambda expressions," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 2017, pp. 60–70.
- [57] "PowerMock framework site." [Online]. Available: <https://powermock.github.io/>
- [58] "JMockit: An automated testing toolkit for java," <https://jmockit.github.io>, accessed: 2025-01-15.